第三章 SQL 注入漏洞与防护——实验分析报告

一. 数据库与查询条件基础说明（任务 1）

以 DVWA 的 SQL Injection 用户 ID 查询为具体场景说明：

1. 页面输入与数据库查询的关联：

页面提供了User ID输入框,用户输入 ID 后,后端需要从数据库的users表中查询该 ID 对应的用户信息（如姓名、密码），再将结果渲染到页面上。页面本身无法存储和查询用户数据，所有用户信息都保存在后端数据库中，因此用户的输入会被后端程序用于构建 SQL 查询语句，向数据库请求对应的数据，最终反馈到页面上。

2. 最小业务链路：

该场景的完整最小业务链路为：用户输入User ID → 后端构建SQL查询语句 → 数据库执行查询并返回结果 → 页面渲染并展示查询结果

3. 用户输入如何进入查询条件：

后端代码会直接将用户输入的User ID拼接到预定义的 SQL 模板中，例如：

后端 SQL 模板：SELECT first_name, last_name FROM users WHERE id = '$input_id'

用户输入1时，$input_id会被替换为用户输入的内容，最终执行的 SQL 语句为：SELECT first_name, last_name FROM users WHERE id = '1'

用户输入的内容会直接成为 SQL 语句中WHERE条件的一部分，参与数据库的查询逻辑。

4. 为什么这为 SQL 注入创造了可能性：

用户输入没有经过校验、过滤或转义，直接被拼接进 SQL 语句，成为了 SQL 语法的一部分，而非仅作为普通的参数值处理。攻击者可以通过输入特殊字符（如'、OR、--）修改 SQL 语句的原有逻辑，例如输入1' OR '1'='1，会将原本的查询条件篡改为恒真条件，让数据库返回全表数据，从而实现 SQL 注入。其核心原因是用户输入污染了 SQL 查询的结构，而非输入中包含的字符本身存在 “危险”。

二. DVWA SQL Injection Low 现象记录（任务 2）

1. 实验页面 / 目标

实验环境：DVWA 本地授权靶场，安全等级设置为 Low，模块为 SQL Injection。

实验目标：观察正常查询与恶意输入的结果差异，验证 SQL 注入现象。

业务逻辑：按 ID 查询用户信息并直接回显。

2. 普通输入与异常输入的对比

（1）普通输入场景

输入内容：1

预期 SQL 语句：SELECT first_name, last_name FROM users WHERE id = '1'

正常结果：仅返回 ID=1 的用户信息：First name: admin, Surname: admin，符合 “按 ID 查询单条用户数据” 的业务逻辑。

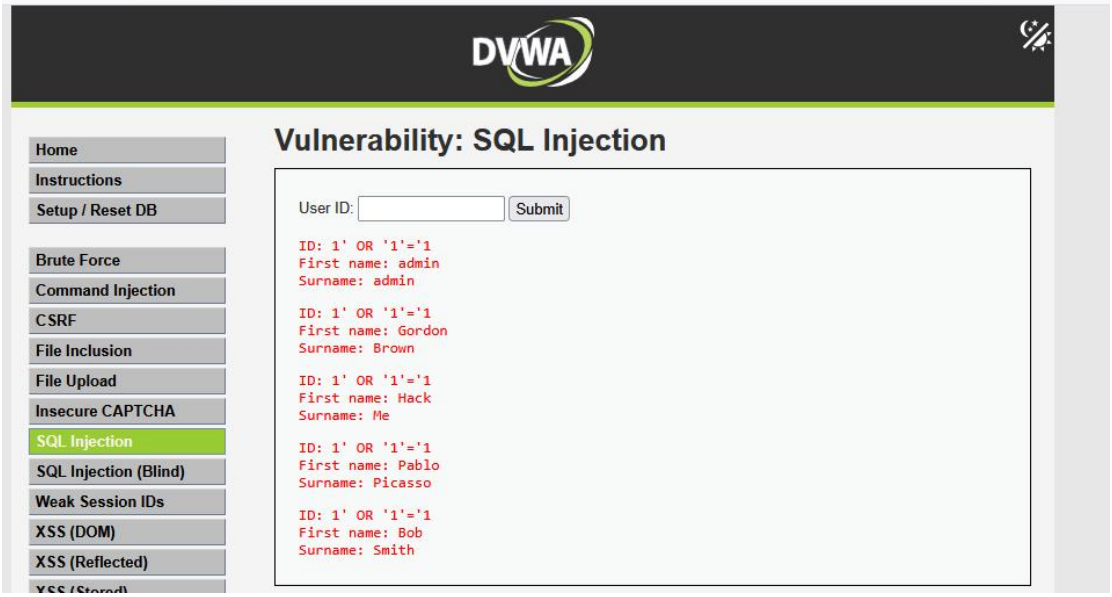
（2）异常输入场景

输入内容：1' OR '1'='1

被篡改的 SQL 语句：SELECT first_name, last_name FROM users WHERE id = '1' OR '1'='1'

截图中的实际结果：页面返回数据库中全部 5 条用户记录，且每条记录的ID字段均显示输入的攻击语句1' OR '1'='1。

截图说明：



3. 返回结果的差异

输入类型	输入内容	页面返回结果差异
普通输入	1	仅返回 ID=1 的单条用户数据，符合业务逻辑预期
异常输入	1’ OR ’1’=’1	返回全表所有用户数据，超出单条查询的业务逻辑，且 ID 字段直接回显了用户输入的攻击语句

4. 为什么这种差异不像 “正常查询变化”

正常查询的固有特征：

正常的 User ID 查询，无论输入合法 ID 还是不存在的 ID，都只会返回与输入 ID 对应的单条数据（或无数据提示），不会返回全表所有用户信息，更不会在结果中直接回显用户输入的内容。

本次差异的本质原因：
输入的'闭合了 SQL 语句中原有的单引号，OR'1'='1 构造了恒为真的条件，将原本的 WHERE id = '1'篡改，导致 SQL 语句的查询逻辑被污染，最终执行了非预期的全表查询。
关键判断依据：
全表数据返回的结果，无法用 "输入参数变化" 解释，只能证明用户输入被解析为 SQL 语法的一部分，而非普通参数值，这是典型的 SQL 注入现象。

三. 机制分析与差异比较（任务 3）

1. 输入如何进入查询结构

完整链路：输入 → 拼接 → 解析 → 执行 → 回显 / 行为差异
输入：用户在 DVWA 的 User ID 输入框提交内容（如 1' OR '1'='1）。
拼接：后端程序未做任何校验，直接将用户输入拼接到预定义的 SQL 模板中（直接拼入 SQL 模板）：
原模板：SELECT first_name, last_name FROM users WHERE id = '$input'
拼接后：SELECT first_name, last_name FROM users WHERE id = '1' OR '1'='1'
解析：数据库将拼接后的字符串作为完整 SQL 语句解析，用户输入的'、OR 被识别为 SQL 语法的一部分，而非普通文本。（数据库将输入当作语法解析）
执行：数据库按篡改后的 SQL 逻辑执行查询，返回全表数据。（执行恒真条件，返回全表）
回显 / 行为差异：页面直接回显所有用户数据，表现出与正常单条查询完全不同的结果。（页面展示全部用户数据）

2. 为什么根因是 "结构被污染" 而不是 "字符本身危险"

'、OR、--等字符本身只是 SQL 语法的组成部分，并非 "天生危险"。
例如，在参数化查询中，输入 1' OR '1'='1 会被作为普通字符串处理，数据库不会将其解析为 SQL 语法，因此不会产生注入效果。
SQL 注入的核心问题是用户输入被错误地解析为 SQL 语法，篡改了查询的结构与逻辑，而非字符本身。这些字符只是攻击者利用语法规则篡改查询结构的工具，而非注入的根本原因。

3. Low 与 Medium 的差异在哪里

维度	Low 等级	Medium 等级
控制思路	完全信任用户输入，无任何校验、过滤或转义，直接拼接用户输入到 SQL 语句	增加了基础的输入校验，对特殊字符（如'）进行转义，或限制输入为数字类型，试图阻断注入的基础路径
防护逻辑	无防护，用户输入可直接修改 SQL 查询结构	仅做表层输入过滤，防护手段单一，无法覆盖所有注入场景（如数字型注入、绕过转义的注入）
注入风险	极易被 SQL 注入攻击，任意含 SQL 语法的输入都可能篡改查询	仍存在注入风险，仅提升了注入门槛，未解决查询结构被污染的根本问题

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

维度	普通 SQL 注入	Blind SQL Injection（盲注）
证据形式	页面直接回显 SQL 语句的执行结果或错误信息，攻击者可直接获取数据	页面不直接回显数据或错误信息，攻击者只能通过页面的响应差异（如是否正常加载、响应时间）判断 SQL 执行结果
攻击	利用回显的结果或报错信息，直	需通过构造条件判断语句，根据页面的行为差异推

维度	普通 SQL 注入	Blind SQL Injection（盲注）
方式	接获取数据库数据	断数据，如基于布尔的盲注、基于时间的盲注
典型场景	DVWA 的 SQL Injection 模块，页面直接返回查询结果	DVWA 的 SQL Injection (Blind) 模块，页面仅显示 “User exists” 或 “MISSING”，无直接数据回显

5. 为什么盲注和普通注入根因相同，但证据形式不同

根因相同：二者的核心问题都是用户输入被解析为 SQL 语法，污染了查询结构，导致数据库执行了非预期的 SQL 语句。

证据形式不同：

普通 SQL 注入中，后端将数据库的查询结果或错误信息直接回显到页面，因此攻击者能直接通过页面数据获取注入证据；

盲注场景下，后端隐藏了查询结果和错误信息，攻击者无法直接获取数据，只能通过页面的行为差异（如响应状态、加载时间）作为注入的间接证据。

强制判断题

SQL 注入的根因更接近：B. 输入进入了不该进入的查询结构

理由：SQL 注入的本质是用户输入被错误地解析为 SQL 语法的一部分，篡改了查询结构，而非输入中包含危险字符。字符只是语法的组成部分，只有当它们被当作 SQL 语法解析时，才会产生注入效果。

Blind SQL Injection 和普通 SQL 注入的最大差异更接近：B. 证据形式不同

理由：二者的根因都是查询结构被污染，差异仅在于页面是否直接回显数据或错误信息，导致攻击者获取注入证据的方式不同，攻击手法因此产生差异。

四. SQL 注入防护清单与解释（任务 4）

SQL 注入的防护分为三层逻辑：

根因控制：解决 “输入被解析为 SQL 语法、污染查询结构” 的根本问题。

暴露面控制：减少攻击者可利用的线索与攻击入口。

后果范围控制：限制注入成功后的影响，降低风险损失。

1. 参数化查询

解决的问题：将用户输入与 SQL 语法完全分离，用户输入仅作为参数值传递给数据库，不会被解析为 SQL 语法，从根源上防止查询结构被污染。

控制层级：根因控制层，直接解决 SQL 注入的核心问题。

不能替代的内容：无法替代输入校验、错误处理、最小权限等防护措施，也无法解决业务逻辑漏洞、数据库配置不当等问题，仅能防护 SQL 注入本身。

2. 输入校验

解决的问题：对用户输入的格式、长度、内容进行校验，过滤或拒绝不符合业务预期的输入（如限制 User ID 仅为数字），减少可被利用的注入输入，降低攻击的可能性。

控制层级：暴露面控制层，减少注入的攻击入口。

不能替代的内容：无法替代参数化查询，复杂场景下的输入校验难以覆盖所有注入绕过方式，不能解决输入被解析为 SQL 语法的根本问题，也无法完全防止恶意输入的绕过。

3. 错误处理

解决的问题：统一错误输出，避免向攻击者暴露数据库结构、字段信息、报错详情等敏感信息，减少攻击者获取的注入线索，阻断普通 SQL 注入的回显证据。

控制层级：暴露面控制层，隐藏注入的攻击证据。

不能替代的内容：无法阻止 SQL 注入本身，只能隐藏错误信息，攻击者仍可通过盲注、时间盲注等方式获取数据，也无法解决查询结构被污染的问题。

4. 最小权限原则

解决的问题：限制数据库账号的权限，例如查询用户仅授予 SELECT 权限，禁止 DROP、ALTER、DELETE 等高权限操作，即使注入成功，攻击者也无法执行高危操作，降低攻击的影响范围。

控制层级：后果范围控制层，限制注入成功后的危害。

不能替代的内容：无法阻止 SQL 注入的发生，只能限制攻击成功后的影响，不能解决查询结构被污染的根本问题，也无法防止数据泄露类的低权限注入攻击。

5. 补充措施：Web 应用防火墙（WAF）

解决的问题：识别并拦截包含 SQL 注入特征的请求（如’ OR ’1’=’1、UNION SELECT 等），在流量层阻断注入攻击，增加攻击的门槛。

控制层级：暴露面控制层，增加攻击的难度与成本。

不能替代的内容：存在被绕过的风险，无法替代参数化查询等根本性防护，也无法应对零日漏洞或新型注入手法，不能解决注入的根本问题。

防护逻辑总结：

SQL 注入的防护需要多层叠加：

根因控制：以参数化查询为核心，解决查询结构被污染的根本问题。

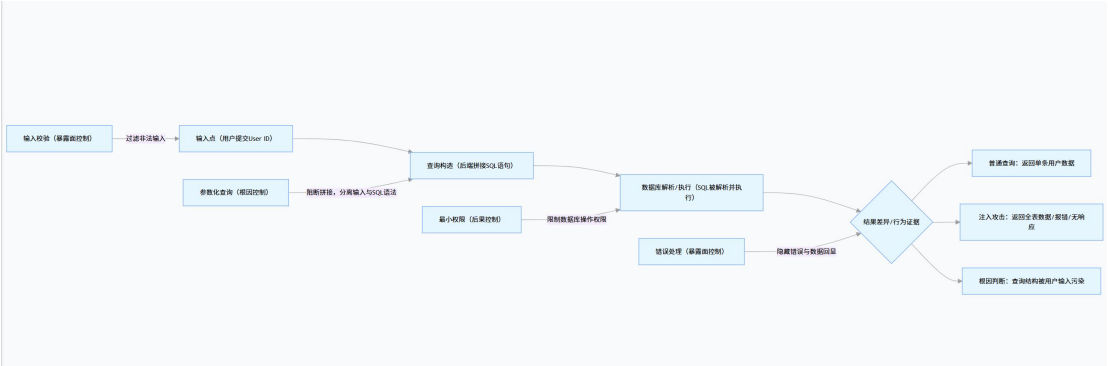
暴露面控制：通过输入校验、错误处理、WAF 减少攻击入口与线索。

后果范围控制：通过最小权限原则降低攻击成功后的影响。

单一防护措施无法完全防御 SQL 注入，多层防护结合才能形成完整的防护链条。

五．收口项

收口项 1：SQL 注入因果链/防护关系图



收口项 2：学生自评表

1.我本章最清楚的一点是什么？

SQL 注入的核心根因是**查询结构被污染**，而非输入中包含特殊字符；参数化查询是解决 SQL 注入的根本措施，能从根源上分离用户输入与 SQL 语法。

2.我最容易混淆的一点是什么？

容易混淆“普通 SQL 注入”与“盲注”的差异，初期误将攻击手法的不同当成根因差异，后来才明确二者根因相同，仅证据形式不同；也容易混淆 Low 与 Medium 等级的防护差异，误将“转义特殊字符”当成根本防护，忽略了其仅为表层输入过滤的局限性。

3.我本次作业中哪一段最需要教师复核？

任务 3 中“Low 与 Medium 等级控制思路差异”的分析，以及任务 4 中各防护措施的控制层级划分，需要教师确认是否准确。

4.如果重做一次，我会优先改哪一部分？

优先完善任务 2 的现象分析，补充更多异常输入（如报错注入、布尔盲注）的测试场景，结合截图详细说明每个差异支撑的判断，避免只描述现象不做分析；同时补充收口项因果链的细节，让防护措施与注入链路的对应关系更清晰。

收口项 3：AI 输出审核与人工复核记录

1.本作业哪些内容使用过 AI 辅助

- 收口项 1 的 SQL 注入因果链 Mermaid 关系图绘制；
- 任务 4 中防护清单的结构化整理与分层说明；

2.哪些判断是你自己复核后保留的

- SQL 注入的根因是“查询结构被污染”，而非特殊字符本身；
- 普通 SQL 注入与盲注的根因相同、证据形式不同的核心结论；
- 各防护措施的控制层级划分（根因/暴露面/后果控制）；
- Low 与 Medium 等级的防护差异分析。

3.哪些内容你发现 AI 输出不准确并进行了修正

- AI 最初将“输入校验”错误归类为“根因控制措施”，经复核修正为“暴露面控制层”；
- AI 对盲注的解释存在偏差，补充了 DVWA 盲注模块的实际场景，明确了“无直接数据回显、仅通过行为差异判断”的核心特征；
- AI 对防护措施的“不能替代什么”部分表述模糊，补充了“参数化查询无法替代输入校验、最小权限”等具体说明。

4.你最终如何确认结论和证据是一一对应的

- 结合 DVWA 本地靶场的实际测试结果，核对每个现象、差异与结论的对应关系，例如输入`1’ OR ’1’=’1`返回全表数据的现象，验证了“查询结构被污染”的结论；
- 对照 SQL 注入的原理，确认各防护措施的控制层级划分准确；
- 对照任务要求的要点，检查所有最低完成要求均已覆盖，无遗漏。

六．提交前自查清单（自评）

- ☑ 仅在授权的 DVWA 本地环境完成实验，未超出授权边界。
- ☑ 所有截图均已脱敏，未暴露敏感信息。
- ☑ 已说明 “正常查询变化” 和 “查询逻辑被改变” 的区别。
- ☑ 已明确 SQL 注入的根因是 “结构被污染”，而非特殊字符。
- ☑ 已区分普通 SQL 注入与盲注的证据形式差异。
- ☑ 已将参数化查询、错误处理、最小权限划分到不同控制层。
- ☑ 已完成 SQL 注入因果链 / 防护关系图。
- ☑ 已填写 AI 输出审核与人工复核记录。

七．红线规则自查

- ☑ 无不脱敏的敏感信息
- ☑ 无超出授权边界的测试行为
- ☑ 实验记录、现象截图与实际过程一致，无伪造证据
- ☑ 作业为独立完成，无抄袭、代做情形